

Structured Priors for the Sparse Tensor Decomposition of Bilinear MLPs

Jabir Hussain

May 24, 2026

1 Introduction

A single-layer bilinear MLP computes its class logits as a quadratic form in the input, $\text{out}_c = \sum_{i,j} B_{c,i,j} x_i x_j$, where B is a third-order interaction tensor. Earlier work decomposes B one class-slice at a time through eigendecomposition [1], but this is constrained to be orthogonal per class and tends to produce superposed, entangled features; structure shared across classes is invisible to it. An alternative factorises the entire tensor at once into R shared components,

$$B_{c,i,j} \approx \sum_{r=1}^R L_{i,r} R_{j,r} D_{c,r}, \quad (1)$$

where each component r carries a left input pattern $L_{:,r} \in \mathbb{R}^{784}$, a right input pattern $R_{:,r} \in \mathbb{R}^{784}$ and class weights $D_{:,r} \in \mathbb{R}^{10}$. The factors are fitted by maximising the cosine similarity between the reconstructed and the true tensor, i.e. minimising $1 - \cos(\hat{B}, B)$, and optimised with Muon [2]; there is no sparsity term and no spatial prior. Using the identity $ab = \frac{1}{4}(a+b)^2 - \frac{1}{4}(a-b)^2$, each component splits into a positive pattern $L_{:,r} + R_{:,r}$ and a negative pattern $L_{:,r} - R_{:,r}$, which we visualise throughout.

The deficiency is structural rather than numerical. Because the 784 pixels enter (1) as an unordered vector, the optimiser has no notion that pixel (5,5) neighbours (5,6), and the recovered patterns are accordingly salt-and-pepper (Figure 1, top). This report asks whether injecting a prior, either on the spatial shape of the components or on their usage across inputs, yields a more interpretable decomposition. All experiments use a bilinear model trained on MNIST to 96.9% test accuracy and a rank-64 factorisation reproducing its tensor at cosine similarity 0.998.

2 Method

Spatial smoothness prior (GP-Sparse). We place a Gaussian-process prior on every component’s input patterns. With a radial-basis kernel over pixel positions, $K_{ij} = \exp(-\|p_i - p_j\|^2 / 2\ell^2)$, the factorisation loss becomes

$$\mathcal{L} = (1 - \cos(\hat{B}, B)) + \lambda \sum_r \left(L_{:,r}^\top K^{-1} L_{:,r} + R_{:,r}^\top K^{-1} R_{:,r} \right). \quad (2)$$

A spatially rough pattern incurs a large quadratic penalty $v^\top K^{-1} v$, whereas a smooth one is nearly free, so the prior pushes each component towards smooth blobs without prescribing their shape. The length-scale ℓ interpolates between two degenerate limits: as $\ell \rightarrow 0$ the kernel approaches the identity and (2) reduces to ridge regression with no spatial coupling, while as $\ell \rightarrow \infty$ the kernel becomes rank-one and every component is forced towards a constant. Two implementation details matter in practice. First, the RBF kernel is severely ill-conditioned, so K^{-1} is formed from its Cholesky factor (`torch.cholesky_inverse`) after a jitter ϵI ; the naive

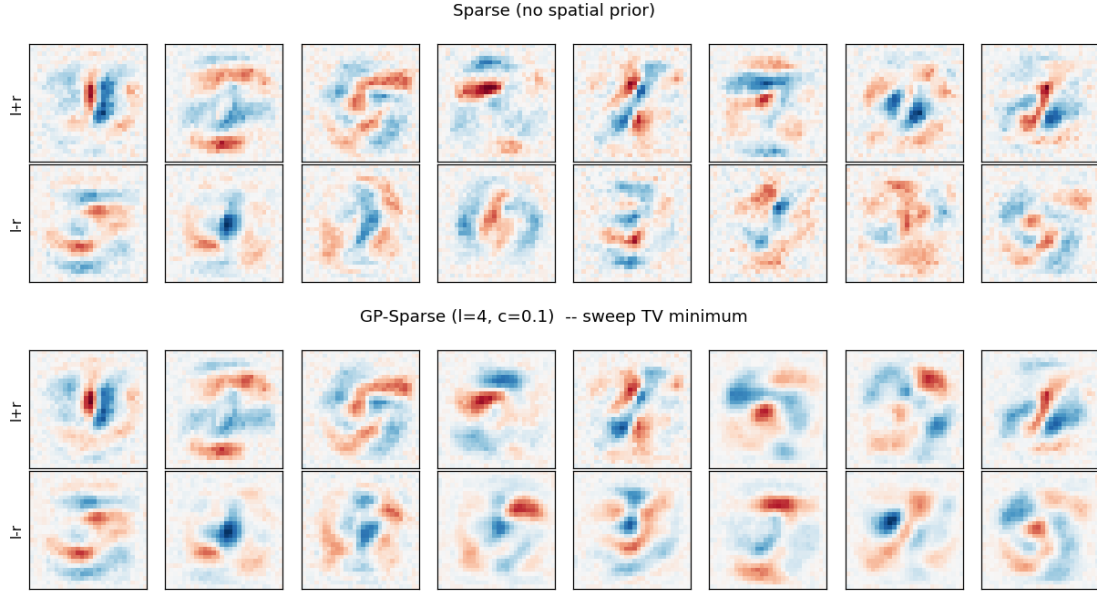


Figure 1: Top-eight components by importance for the baseline (top) and the GP-Sparse decomposition at $\ell = 4$ (bottom). Each pair of rows shows the positive ($l+r$) and negative ($l-r$) input patterns. The spatial prior removes the high-frequency speckle and yields cleaner strokes, but the components remain qualitatively the same kind of blob.

`torch.linalg.inv` returns a non-symmetric, numerically degenerate matrix (symmetric error ~ 40). Second, the raw penalty scales as $1/\epsilon$, so a fixed λ is uninterpretable; we instead anchor it as $\lambda = c/\text{anchor}$, where the anchor is the penalty evaluated at the unregularised solution, so that c is a dimensionless regularisation pressure that is comparable across length-scales.

Activation-sparsity prior (ActSparse). Following the criterion behind Attribution-based Parameter Decomposition [3], a good decomposition should be sparsely used: for any single input only a few components should be active. Each component’s activation is $a_r(x) = (L_{:,r}^\top x)(R_{:,r}^\top x)$, and we add an L_1 penalty on its mean absolute value over the training data,

$$\mathcal{L} = (1 - \cos(\hat{B}, B)) + \lambda \mathbb{E}_x \left[\frac{1}{R} \sum_r |a_r(x)| \right]. \quad (3)$$

Without a constraint this is trivially evaded by shrinking L, R and inflating D , since (1) is multilinear and the activation is not. We therefore project each column of L, R to unit norm after every optimiser step and absorb the scale into D ; this preserves the tensor exactly but fixes the gauge in which the penalty is measured. As above, λ is anchored so that c equals the L_1 term evaluated at the (gauge-fixed) unregularised solution. Although Muon is a smooth-update optimiser and the L_1 term is non-smooth, training remained stable once the unit-norm constraint was enforced, and no optimiser change was required.

3 Results and Discussion

3.1 The smoothness prior denoises for free, but reveals nothing

The GP prior behaves exactly as intended: the salt-and-pepper texture thins out and the strokes become smoother (Figure 1). Quantitatively, the headline run ($\ell = 4$, $c = 0.1$) reduces the mean total variation of the normalised components from 0.0372 to 0.0263, a 29% drop, while reconstruction similarity falls only from 0.998 to 0.995 and test accuracy is unchanged at 96.8%. A length-scale sweep (Figure 2) confirms the expected behaviour at the limits and locates

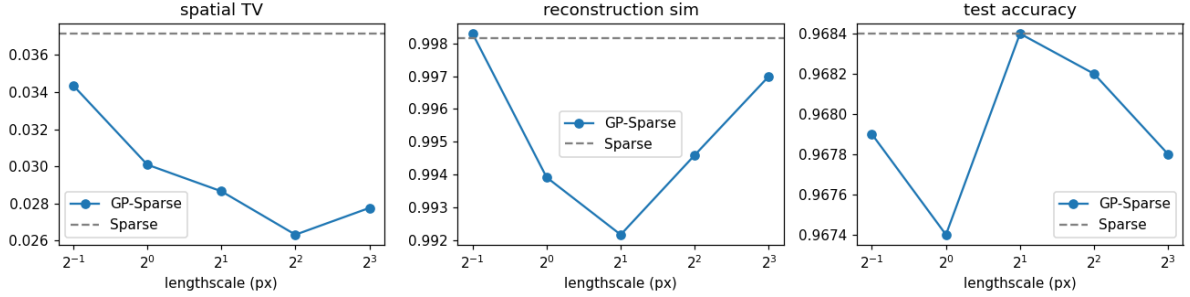


Figure 2: Length-scale sweep at fixed regularisation pressure $c = 0.1$. Total variation is minimised near $\ell = 4$, where reconstruction similarity is also highest among the regularised runs; test accuracy is essentially flat across the entire range. The dashed line is the unregularised baseline.

Model	$\cos(\hat{B}, B)$	Test acc.	L0@1%	L0@5%	Dead frac.
Sparse (baseline)	0.998	0.968	43.6	17.5	0/64
ActSparse, $c=0.03$	0.997	0.964	39.5	13.7	0/64
ActSparse, $c=0.1$	0.992	0.951	35.5	12.1	2/64
ActSparse, $c=0.3$	0.978	0.926	27.0	8.6	8/64
ActSparse, $c=1.0$	0.936	0.808	16.0	3.9	18/64

Table 1: Activation-sparsity sweep. L0@ τ is the mean number of components whose $|a_r(x)|$ exceeds τ of the global maximum on a given input; dead fraction counts components never reaching 1% of that maximum anywhere in the dataset. Every quantity degrades monotonically with c .

a mild optimum near $\ell = 4$, which dominates the conventional $\ell = 2$ default on every axis. A σ -weighted total variation, which weights each component by its importance to the reconstruction, is almost identical to the unweighted figure, indicating that the smoothing is uniform across components rather than an artefact of averaging over unimportant noise directions.

The interpretation, however, is sobering. That one can remove a quarter of the spatial roughness at almost no cost to reconstruction or accuracy implies that the texture lived in the null space of the factorisation objective; it was unconstrained noise that the loss never penalised rather than hidden structure that the baseline was failing to express. The base model was moreover trained with Gaussian-noise augmentation, which already biases the underlying tensor towards smoothness, so the speckle is largely an artefact of giving the decomposition 784 free parameters per pattern. The importance spectrum is unchanged in shape, and no low-rank or modular organisation emerges. The GP prior therefore tidies the visualisation without delivering new interpretability.

3.2 The sparsity prior is costlier, but surfaces real features

The sparsity prior tells a harsher quantitative story (Table 1, Figure 3). In contrast to the smoothness prior, every metric degrades monotonically with c : there is no cost-free regime, and L0 sparsity trades against accuracy almost linearly. The headline setting $c = 0.1$ already costs 1.7 points of test accuracy for a 31% reduction in L0@5% and only two dead components; reaching a meaningfully sparse solution (18/64 dead at $c = 1.0$) collapses accuracy to 80.8%. The components also remain spatially rough, since the prior says nothing about shape, and the importance spectrum flattens rather than developing a low-rank hierarchy (Figure 5).

The qualitative payoff is nonetheless real. Inspecting the strongest-activating training images per component (Figure 4) shows that roughly three of the top eight components are genuinely monosemantic: component 20 fires exclusively on zeros, and components 13 and 23 are clearly tuned to “2” and “4”. These are precisely the reusable, single-concept building blocks that

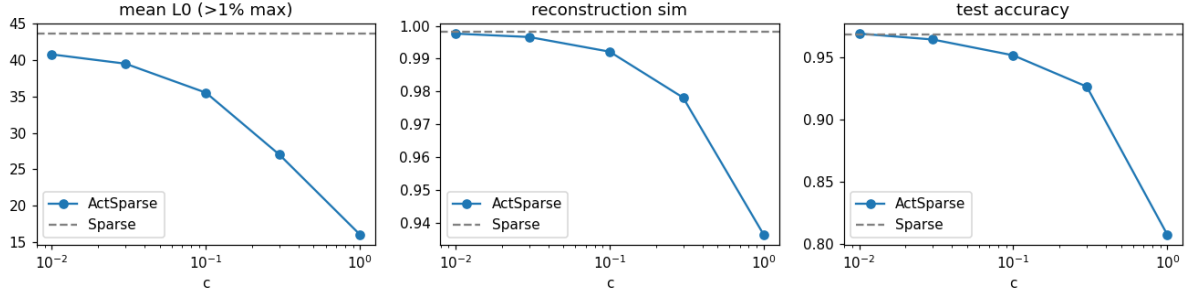


Figure 3: Activation-sparsity sweep. Unlike the smoothness prior, there is no cost-free regime: mean L0, reconstruction similarity and test accuracy all fall together as c increases.

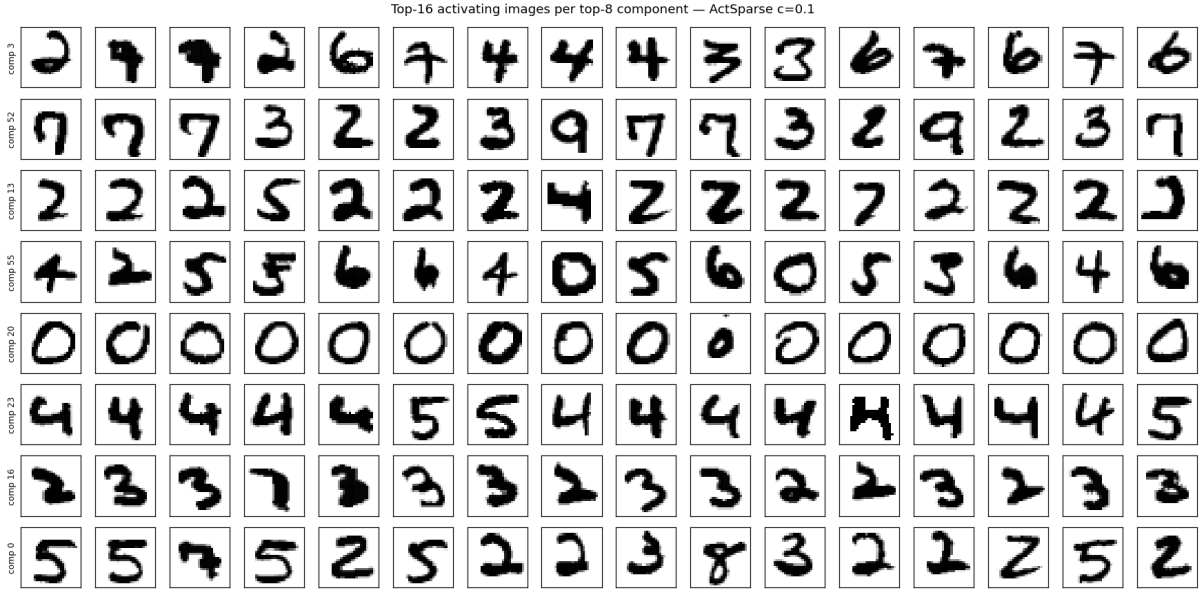
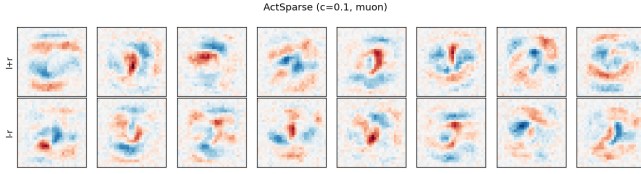


Figure 4: The interpretability test, borrowed from the sparse-autoencoder literature [4]: the sixteen training images that most strongly activate each of the top-eight components ($c = 0.1$). Component 20 is a clean “0 detector” (16/16 zeros); components 13 and 23 are recognisably “2” and “4” selective. The remaining rows stay polysemantic, mixing several digits.

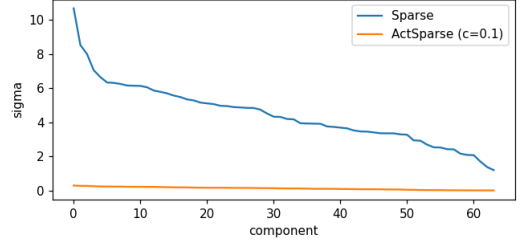
the eigendecomposition and the unregularised baseline failed to isolate. The criterion does not cleanly disentangle the network, since over half of the top components remain polysemantic mixtures, but it surfaces a handful of honestly interpretable features that were not previously accessible.

4 Conclusions

The two priors fail and succeed in instructive ways. The Gaussian-process smoothness prior produces a cleaner picture almost for free, but its cheapness is diagnostic: it removes noise that the reconstruction objective never constrained, and delivers no new interpretable structure. The activation-sparsity prior is the more faithful of the two to the stated goal of interpretability, isolating a small set of genuinely monosemantic components (including a clean “0 detector”), but it is not free, trading accuracy for sparsity roughly linearly and disentangling the network only partially. Taken together, the experiments suggest that priors on the shape of mechanistic components are cosmetic for this model, whereas priors on their usage are where the interpretability signal lies.



(a) Components remain salt-and-peppery.



(b) Importance spectrum flattens.

Figure 5: ActSparse side-effects ($c = 0.1$). The activation-sparsity prior leaves the spatial texture untouched (a) and removes the baseline’s importance hierarchy (b), under the unit-norm gauge in which all components carry comparable magnitude.

5 Future Work

The activation-sparsity result points to the most promising direction. Our L_1 penalty on $|a_r(x)|$ is only a first-order proxy for the criterion that Attribution-based Parameter Decomposition (APD) actually proposes [3]. Rather than penalising raw activation magnitude, APD scores each component by its causal attribution to the output, and decomposes a network into mechanisms that are simultaneously faithful (they reconstruct the original model), minimal (few are active on any given input) and simple (each occupies as few ranks as possible). Our ActSparse objective captures only the minimality term and approximates it crudely; replacing the activation-magnitude penalty with APD’s gradient-based attribution importance, and adding the faithfulness and simplicity terms, is the natural next step and would test directly whether the polysemantic components that our proxy left entangled (Figure 4) can be cleanly separated.

Three further directions follow. First, the two priors are complementary and could be combined, yielding components that are at once sparsely used and spatially coherent, which may improve human readability without the accuracy cost of activation sparsity acting alone. Second, the GP null-result should be retested on a base model trained without Gaussian-noise augmentation, where the high-frequency texture may carry genuine signal rather than noise, to establish whether the smoothness prior is truly cosmetic or merely redundant with the teacher’s own regularisation. Third, both priors should be scaled beyond MNIST and rank-64 to determine whether monosemantic components proliferate with capacity, or whether the accuracy-sparsity tradeoff sharpens.

Code availability

The two priors are implemented as drop-in model classes, `src/image/gp_sparse.py` and `src/image/act_sparse.py`, extending the bilinear-decomposition codebase of Pearce et al. [1]. All experiments, sweeps and figures are reproduced in the notebook `exercises/0_decomposition.ipynb`. The repository is available at <https://github.com/j-hussain/bilinear-decomp>.

References

- [1] M. T. Pearce, T. Dooms, A. Rigg, J. Oramas, and L. Sharkey. *Bilinear MLPs enable weight-based mechanistic interpretability*. arXiv:2410.08417, 2024.
- [2] K. Jordan. *Muon: An optimizer for the hidden layers of neural networks*. Technical report, 2024.

- [3] D. Braun, L. Bushnaq, S. Heimersheim, J. Mendel, and L. Sharkey. *Interpretability in parameter space: Minimizing mechanistic description length with attribution-based parameter decomposition*. arXiv:2501.14926, 2025.
- [4] T. Bricken et al. *Towards monosemanticity: Decomposing language models with dictionary learning*. Transformer Circuits Thread, 2023.
- [5] C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.